MEMBER 1 • DATA PACKAGES, DAO INTERFACES & PERSISTENCE

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (POINT THESE OUT TO SIR!)

📦 **Packages:** `com.cricbid.db`, `model`, `dao`, `dao.impl`

🔌 **Interfaces:** 6 DAO interfaces (`TeamDAO`, `PlayerDAO`...) + `Serializable`

📚 **Java Libraries:** `java.sql.*`, `java.util.Properties`, `java.io.FileInputStream`, `java.security.MessageDigest`

🛡️ **Exception Handling:** `try-with-resources`, `SQLException`, `ClassNotFoundException`, `AuthenticationException`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"Good afternoon, Sir. I'll explain CricBid's data packages, DAO interfaces, Java libraries, and exception handling, along with our Review 2 multi-laptop upgrades.

Our project is organized into ten modular packages. My layer spans `com.cricbid.db`, `model`, `dao`, and `dao.impl`, enforcing a strict three-tier architecture over six MySQL tables: `teams`, `users`, `players`, `squads`, `bids`, and `auction_log`.

For **interfaces**, we define six DAO contracts—such as `PlayerDAO`, `TeamDAO`, and `BidDAO`—implemented in `dao.impl` using `java.sql.PreparedStatement` to prevent SQL injection. Using **Java libraries**, `DBConnection` uses `java.util.Properties` and `java.io.FileInputStream` inside a thread-safe Double-Checked Locking Singleton, while `AuthService` uses `java.security.MessageDigest` for SHA-256 password hashing. When mapping rows, `PlayerFactory` instantiates `Batsman`, `Bowler`, `AllRounder`, or `WicketKeeper`, each overriding `calculateValue()`. For **exception handling**, every DAO uses `try-with-resources` to close connections safely, handles `SQLException`, and `AuthService` throws our custom `AuthenticationException` on invalid logins.

For Review 2, we added `DBConnection.configureHost()` so client laptops connect to Laptop 1's IP, added `updateOwnerType()` to the `TeamDAO` interface, created `lan_setup.sql` and `owner1` through `owner6` in `seed.sql`, and implemented the `java.io.Serializable` interface across all model classes for network transfer."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
Package (`package com.cricbid.dao;`)	Like cleanly labeled folders for Java classes (`db`, `dao`, `dao.impl`, `model`). Prevents naming mix-ups and enforces modular architecture.
Interface (`TeamDAO`) & `implements`	An **interface** is a pure contract/checklist of methods with NO SQL inside. `TeamDAOImpl implements TeamDAO` provides the actual SQL code for every method on that checklist.

`Serializable` Interface	A built-in `java.io.Serializable` marker interface that gives Java permission to pack `Player`, `Team`, and `Bid` objects into bytes so they can travel over Wi-Fi sockets!
Java Standard Libraries	Ready-made official packages built into Java: `java.sql` (JDBC), `java.util.Properties` (config files), `java.security.MessageDigest` (SHA-256 encryption).
`try-with-resources` Exception Handling	Writing `try (Connection conn = ...)` guarantees Java **automatically closes** the database connection even if an SQL error crashes mid-query!
`Singleton` + `synchronized`	`Singleton` ensures only **ONE** `DBConnection` object ever exists. `synchronized` acts like a door lock so two threads can't create two copies at startup.

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `DBConnection.java` — Package `com.cricbid.db`, Libraries & Singleton

📂 Open: src/main/java/com/cricbid/db/DBConnection.java (Lines 1–34 & Lines 84–91)

```
package com.cricbid.db;
import java.io.FileInputStream;      // Java I/O Library
import java.sql.DriverManager;       // Java SQL Library
import java.util.Properties;         // Java Util Library

public class DBConnection {
    private static volatile DBConnection instance;
    private final Properties properties = new Properties();

    private DBConnection() { loadProperties(); } // Private constructor = Singleton!

    public static DBConnection getInstance() {
        if (instance == null) {                    // 1st check (fast, no lock)
            synchronized (DBConnection.class) {    // Thread lock
                if (instance == null) {            // 2nd check (inside lock)
                    instance = new DBConnection();
                }
            }
        }
        return instance;
    }
    ...
    // NEW IN REVIEW 2 (Lines 84–91): Dynamic LAN Host IP!
    public synchronized void configureHost(String host) {
        String cleanHost = (host == null || host.trim().isEmpty()) ? "localhost" : host.trim();
        this.activeHost = cleanHost;
        String newUrl = String.format("jdbc:mysql://%s:3306/cricbid?...", cleanHost);
        properties.setProperty("db.url", newUrl);
    }
```

- **Java Libraries & Exception Handling in `loadProperties()` (Lines 40–82):** Uses `Properties` + `FileInputStream` inside `try-with-resources`, catching `IOException` when reading `db.properties` and catching `ClassNotFoundException` when loading the MySQL driver (`Class.forName(...)`).
- **Review 2 Upgrade (`configureHost`):** Rewrites `db.url` from `localhost` to Laptop 1's Hotspot IP (`192.168.x.x:3306`) when Laptops 2 & 3 log in.

File 2: `TeamDAO.java` & `TeamDAOImpl.java` — Interface + `try-with-resources`

📂 Open: src/main/java/com/cricbid/dao/TeamDAO.java & dao/impl/TeamDAOImpl.java (Lines 72–81)

```
// 1. Interface in package com.cricbid.dao
public interface TeamDAO {
    List<Team> findAll() throws SQLException;
    boolean updateOwnerType(int teamId, OwnerType ownerType) throws SQLException; // NEW in Review
2!
}

// 2. Implementation in package com.cricbid.dao.impl
public class TeamDAOImpl implements TeamDAO {
    @Override
    public boolean updateOwnerType(int teamId, OwnerType ownerType) throws SQLException {
        String sql = "UPDATE teams SET owner_type = ? WHERE team_id = ?";
        try (Connection conn = db.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, ownerType != null ? ownerType.name() : OwnerType.BOT.name());
            stmt.setInt(2, teamId);
            return stmt.executeUpdate() > 0;
        }
    }
}
```

Exact Words to Say to Sir (Hits ALL Rubric Points!):

"Sir, notice how we separated the interface in `com.cricbid.dao` from the SQL implementation in `com.cricbid.dao.impl`. Inside `TeamDAOImpl`, we use `java.sql.PreparedStatement` with `?` placeholders to block SQL injection, and `try-with-resources` so `conn` and `stmt` close automatically even if `SQLException` is thrown."

File 3: `AuthService.java` — `java.security` Library & Custom `AuthenticationException`

📂 Open: src/main/java/com/cricbid/service/AuthService.java (Lines 27–58)

```
public static String hashPassword(String password) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256"); // java.security library!
        byte[] hash = md.digest(password.getBytes(StandardCharsets.UTF_8));
        ...
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("SHA-256 algorithm unavailable", e);
    }
}

public User login(String username, String password) throws AuthenticationException, SQLException {
    if (username == null || username.trim().isEmpty() || password == null || password.isEmpty()) {
        throw new AuthenticationException("Username and password cannot be empty.");
    }
    User user = userDAO.authenticate(username.trim(), hashPassword(password));
    if (user == null) {
        throw new AuthenticationException("Invalid username or password.");
    }
    return user;
}
```

- **Java Library (`java.security.MessageDigest`):** Converts plain passwords (`owner123`) into irreversible 64-character SHA-256 hex hashes.
- **Custom Exception Handling:** Catches `NoSuchAlgorithmException` and throws our custom checked `AuthenticationException` (`com.cricbid.exception`) when login fails.

File 4: `Player.java` & `PlayerFactory.java` — `Serializable` Interface & OOP

📂 Open: src/main/java/com/cricbid/model/Player.java (Line 7) & PlayerFactory.java (Lines 17–28)

```
public abstract class Player implements Serializable { // Built-in java.io.Serializable interface!
    private static final long serialVersionUID = 1L;
    public abstract double calculateValue();
    public abstract String describe();
}
```

- **Interface (`implements Serializable`) NEW IN REVIEW 2 :** Added to `Player` , `Team` , `Bid` , `SquadEntry` , and `User` so Java sockets can stream these objects across Wi-Fi without throwing `NotSerializableException` .
- **OOP + Factory:** `Batsman` , `Bowler` , `AllRounder` , and `WicketKeeper` extend `Player` and override `calculateValue()` , created dynamically by `PlayerFactory` based on the SQL `role` column.

File 5: `lan_setup.sql` & `seed.sql` NEW IN REVIEW 2

- `lan_setup.sql` grants `'cricbid'@'%'` remote LAN access; `seed.sql` adds `owner1` .. `owner6` for multi-laptop logins.

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: Why did you put `TeamDAO` in `com.cricbid.dao` and `TeamDAOImpl` in `com.cricbid.dao.impl`?

Answer: "Sir, separating interfaces from implementations across packages enforces loose coupling. Service classes only depend on the `TeamDAO` interface contract, never on raw SQL classes."

Q2: Why did you implement the `Serializable` interface on model classes in Review 2?

Answer: "`java.io.Serializable` is a marker interface that allows `ObjectOutputStream` to convert our `Player`, `Team`, and `Bid` objects into byte streams over TCP sockets. Without it, Java throws `NotSerializableException`."

Q3: How does `try-with-resources` help in exception handling?

Answer: "Because `Connection` and `PreparedStatement` implement `AutoCloseable`, `try (...)` guarantees they close automatically even if `SQLException` occurs, preventing database connection leaks."